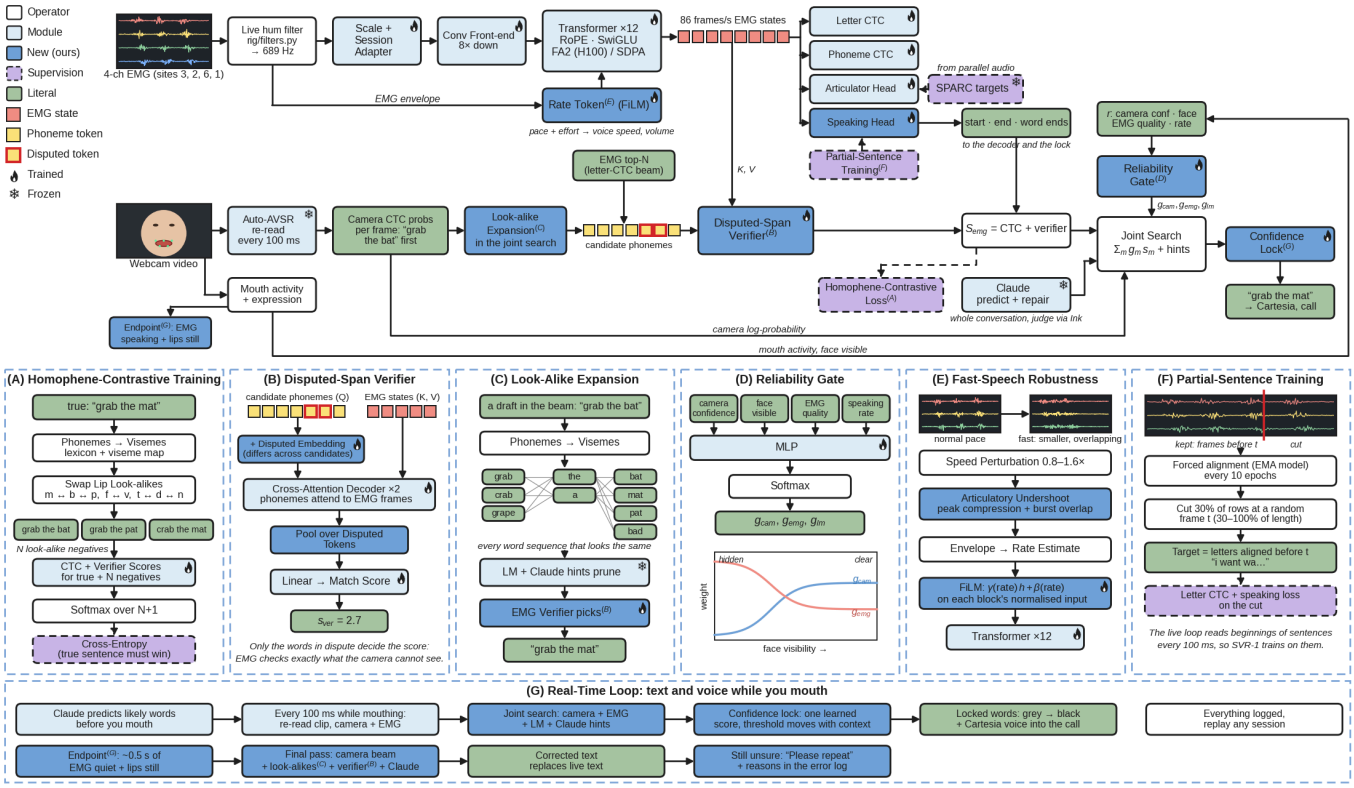
SVR-1: engineering spec

SVR-1 is our EMG model: it reads 4 channels of face-muscle signals and, every 100 ms, gives the fusion decoder per-frame letter and sound probabilities, a speaking-or-not signal, the speaking pace and effort, and a verifier that settles disputed look-alike words. This page specifies it precisely enough to implement: inputs, layers, outputs, losses, training, live use, and the models that sit on top of it (the decoder and the lock model). How it fits the whole system: REALTIME_PIPELINE.md.

The code is in `svr/` (model, training, fine-tuning), `bench/` (benchmarks), `cluster/` (H100/H200 jobs) and `tests/`. It currently implements the previous design; the last section lists every change.



The figure is drawn by `figures/make_architecture.py` (also as `figures/svr1_architecture.pdf`).

1. Input

Electrodes. 4 BioAmp EXG Pill channels at Gaddy and Klein's sites 3, 2, 6, 1 (their best 4-electrode set), read by the Uno at 1,000 Hz, 10-bit (0 to 1023). Rig columns arrive in pin order A0, A1, A2, A3.

Channel slots. The model has 8 input slots in Berkeley's channel order (slot = Gaddy site − 1), so Berkeley pretraining and our rig share one model. Our 4 channels go to slots 2, 1, 5, 0; the other 4 slots are zero and masked.

Rig pin	Gaddy site	Where	Model slot
A0	3	Under the chin	2
A1	2	Left corner of the chin	1
A2	6	Right cheek, below the mouth	5
A3	1	Left cheek, above the mouth	0

In code: `RIG_TO_SLOT = [2, 1, 5, 0]`, `KEEP = [0, 1, 2, 5]` (the existing `4ch` channel set in `bench/common.py`).

Filtering: one filter everywhere. Every EMG signal the model sees, live, in our recordings, and in Berkeley's pretraining data, goes through Thai's causal filter `rig/filters.py`: a 4th-order Butterworth high-pass at 20 Hz plus notches at 60, 120 and 180 Hz (Q = 10), applied with `scipy.signal.sosfilt` in time order, its state initialised from the first sample and carried across chunks. Never `filtfilt`. Berkeley's raw 1 kHz recordings are re-filtered this way and re-cached; Gaddy's original zero-phase notch and 2 Hz high-pass are no longer used.

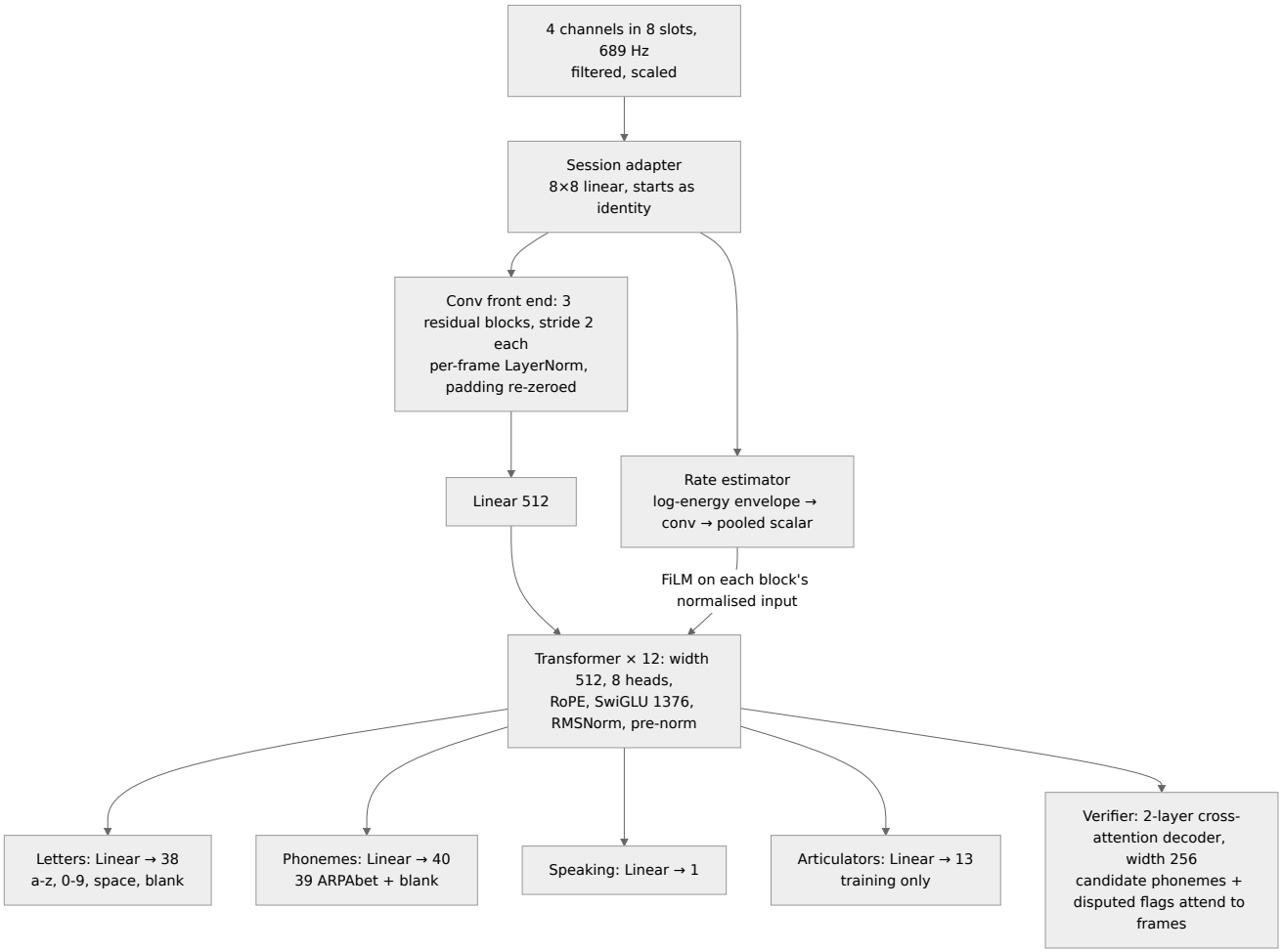
Resampling and frames. As in Gaddy and Klein: linear resampling from 1,000 Hz to 689.06 Hz, then frames of 8 samples, so 86.13 frames per second (11.6 ms per frame).

Scaling. Our ADC counts and Berkeley's units differ, so each channel is scaled to Berkeley's level before the usual soft clip:

1. Per channel, the robust spread of the filtered signal over a session is `1.4826 × median(|x − median(x)|)`, computed over all of that session's clips (live: over the calibration minute).
2. Multiply channel *c* by `s_c = target_c / spread_c`, where `target_c` is the same statistic over Berkeley's filtered data for that site (stored once in `data/svr1_targets/scale.json`).
3. Then Gaddy's soft clip: `x / 20`, then `50 · tanh(x / 50)`.

The session adapter (below) learns whatever difference remains.

2. Network



- **Session adapter:** an 8×8 linear map applied per sample, initialised to identity, with no weight decay; channel mask applied before and after it.
- **Conv front end:** 3 residual blocks, each a stride-2 1D convolution (kernel 3) + per-frame LayerNorm + GELU + a second convolution, with a 1×1 strided shortcut. 689 Hz in, 86 Hz out (8× down). Padding is re-zeroed after every block, so a clip gives the same output alone or in a batch, and a growing clip in the live loop gives stable outputs for the frames it already had.
- **Rate estimator:** per-frame log energy of each slot → two 1D convolutions → mean over valid frames → one number: the predicted speaking rate (normalised log letters per second).
- **Transformer:** 12 pre-norm blocks, width 512, 8 heads, rotary position embeddings, SwiGLU feed-forward (hidden 1376), RMSNorm, dropout 0.1, residual stream in fp32, matmuls in bf16. Attention never crosses utterances (key-padding mask). FlashAttention-2 with variable-length packing on H100/H200; PyTorch SDPA on the RTX 5090 and the laptop.
- **Rate FiLM:** in every block, a zero-initialised linear map turns the rate estimate into a scale and shift applied to the normalised inputs of the attention and feed-forward branches (adaLN style), never to the residual stream.

- **Size:** about 45M parameters. The reference speed for this size is the Gaddy-architecture model at 3.9 ms per 3 s clip on an H200; SVR-1 is a similar size and gets measured on the RTX 5060 (target: under 10 ms per pass for a 5 s window, fp16).

3. Outputs

Output	Shape	Used for
Letter log-probabilities	frames × 38	Proposing words (CTC prefix search) and grading any candidate (CTC log-likelihood) inside the joint search; the space symbol marks word ends
Phoneme log-probabilities	frames × 40	Grading candidates by sound (CTC log-likelihood), used only when every candidate has a pronunciation
Speaking probability	frames	Sentence start and end, word boundaries, gating the camera's activity signal
Rate	1 per utterance	The voice's speed (`generation_config`), and the gate
Effort	frames	Not learned: RMS of the filtered, scaled signal over 100 ms, normalised by the session's speaking median; the voice's volume
Verifier score	1 per candidate	Settling the few disputed words between close candidates
Articulator positions	frames × 13	Training only: 12 tongue/lip/jaw trajectories + loudness from SPARC

Candidate score. For candidate sentence *y*, SVR-1's score is

```
S_{emg}(y) = w_L \log P_{CTC}(y_{\text{letters}} \mid x) + w_P \log P_{CTC}
(y_{\text{phonemes}} \mid x) + w_V\, v(y)
```

with (w_L, w_P, w_V) = softmax of three learned numbers (they always sum to 1). The phoneme term is dropped for the whole candidate set if any candidate lacks a pronunciation. A candidate too long for the frames (no CTC path) scores −10,000. Pronunciations come from CMUdict, indexed with apostrophes removed, plus a g2p_en supplement for names (`data/lexicon/g2p_supplement.dict`, 99.4% of Berkeley sentences covered).

4. Training

Losses

Loss	Weight	Target
Letter CTC	1.0	The sentence's characters (lowercase, digits, single spaces)
Phoneme CTC	0.5	ARPAbet phonemes; rows without a full pronunciation are skipped
Articulator MSE	0.5	SPARC's 13 trajectories from the voiced Berkeley audio, resampled to frames; silent recordings get their voiced twin's targets aligned by DTW on EMG energy

Loss	Weight	Target
Rate MSE	0.1	(log letters per second − 2.34) / 0.37 after augmentation
Speaking BCE	0.2	1 from the first to the last non-blank letter of the forced alignment, widened by 50 ms on each side; 0 elsewhere; rest clips all 0
Judge cross-entropy	0.5, from step 3,000, ramped over 3,000 steps	The true sentence must out-score K = 8 negatives under S_emg

The judge loss (homophene-contrastive training) runs on 8 utterances per micro-batch. Negatives are the sentence with words swapped for lip look-alikes (same viseme sequence, from CMUdict + a viseme map, vocabulary from the training sentences only). On Austin's data, the camera's own wrong guesses are added as negatives: Auto-AVSR's 8-best on the recorded video of each clip.

Partial-sentence training. The live loop reads the beginning of a sentence every 100 ms, so SVR-1 learns from beginnings:

1. From epoch 20, every 10 epochs, compute a forced alignment of every training utterance with the current EMA model (Viterbi over the letter CTC), and cache it.
2. For 30% of rows in each batch, cut at a random frame t between 30% and 100% of the length.
3. The target is the characters aligned before t, a partial last word included.
4. Letter CTC and the speaking loss train on the cut; phoneme, articulator and judge losses skip those rows.

The same alignments give the speaking targets.

Augmentation, on the fly, reproducible from (seed, epoch, batch): - **Speed** 0.8 to 1.6× with probability 0.7, applied only if the target still fits. - **Articulatory undershoot** with probability 0.6 when sped up: compress burst peaks by up to 35% and overlap neighbouring bursts by up to 35%, with a 10 to 40 ms shift. - **Hardware simulation** with probability 0.8: small channel mixing, gain ±15%, a random first-order high-pass between 2 and 70 Hz half the time, 10-bit quantisation, noise. - **Time masking:** up to 3 spans of up to 8 frames. - **Channel dropout**, only in 8-slot pretraining: p = 0.25, at least 3 channels kept.

Optimisation: - AdamW, betas 0.9 / 0.98, learning rate 4e-4, weight decay 0.01 (none on norms, biases and the adapter). - 1,000 warm-up steps, cosine decay to 5%, gradient clipping at 1.0. - EMA of weights 0.999, with a warm-up of `min(0.999, (1 + step) / (10 + step))`. - bf16. - Batches: one utterance per row, sorted within chunks of 1,024 and filled to 112,000 padded samples (about 97% real signal, about 405 steps per epoch). - 150 epochs, about 61,000 steps.

Stages

Stage	Data	Trains	Notes
0. Targets	Berkeley (19 h, 1 speaker)	nothing	Re-filter and re-cache with `rig/filters.py`; SPARC articulators; phonemes; scale statistics

Stage	Data	Trains	Notes
1. Pretrain	Berkeley, slots 0, 1, 2, 5	everything	Main run `SVR1_4ch`; optional 8-slot run with channel dropout, then 50 epochs on the 4 slots (`--init-from`). Cluster: H100/H200
2. Fine-tune	Austin, sessions 1 and 2	everything, lr 1e-4, 40 epochs, EMA 0.998	Rest clips as negatives for speaking; camera N-best negatives for the judge; 10% of session 2 held out for early stopping
3. Calibrate	~1 min before the demo	adapter only, 50 steps	Also recomputes the scale factors s_c

The comparison baseline is Gaddy and Klein's architecture retrained on the same 4 slots with their recipe (`G4`), on the Berkeley benchmarks and on Austin's session 3.

5. Live use, every 100 ms

1. The EMG reader keeps the filtered, scaled signal of the current segment in a ring buffer. The filter is causal, so it runs once per sample as samples arrive; nothing is re-filtered.
2. Every 100 ms, SVR-1 re-reads the whole current segment (up to 8 s; longer segments keep the last 2 s as context) and publishes per-frame letter and phoneme log-probabilities, the speaking probability, the rate and the effort.
3. The decoder calls the verifier only for the disputed candidates of words about to lock.
4. A segment starts when the speaking probability (or the camera's mouth activity) rises, including 300 ms before, and ends after 0.5 s below threshold on both.

6. The models on top of SVR-1

These are not part of the network but are trained on the same data, so they are specified here.

The joint decoder

A beam search (beam 16) over the camera's word pieces (Auto-AVSR's SentencePiece vocabulary, top 32 tokens per frame). Every partial hypothesis h carries four running scores, updated incrementally as it grows:

```
S(h) = g_{cam} \log P^{cam}_{CTC}(h) + g_{emg} \log P^{emg}_{CTC}(\text{chars}(h)) + g_{lm}
\log P_{LM}(h) + \beta \cdot \text{hints}(h) + \gamma \, |h|
```

- **Camera and EMG terms:** CTC prefix probabilities, the camera's over word pieces at 25 fps and SVR-1's over characters at 86 fps. A hypothesis is converted to characters by lowercasing it and turning "▁" into a space.
- **LM term:** a word-level KenLM n-gram language model.
- **Hints:** a bonus for each word in Claude's current prediction list, weighted by its prediction weight.
- **Length term:** γ|h| is a length bonus.

- **Weights:** g_cam, g_emg, g_lm come from the gate.

Extra hypotheses enter the beam from two places: SVR-1's own best letter strings, and look-alike expansion, where each finished word is also tried as its viseme-identical alternatives.

The gate is a softmax over (camera, EMG, LM) from a small linear layer. Its inputs are: - camera confidence (mean top probability, over the last 0.5 s); - the share of recent frames with a clearly visible face; - mouth activity and EMG speaking probability; - EMG quality (clipping and hum from the signal check's measures); - rate.

It is fitted on sessions 1 and 2 by making the true sentence win among the beam's 16-best: the same objective as the judge loss, with a few dozen parameters.

The lock model

After each pass, every unlocked word gets one probability that it is right and final:

```
p_{lock}(w) = \sigma(\theta \cdot f(w))
```

The features f(w) are: - the decoder's posterior for the word at its position, across the beam; - the camera-only and EMG-only posteriors, after rescoring the beam with one reader alone; - their minimum (agreement); - Claude's prior for the word, capped at 0.3; - the boundary probability: the next word started, the space posterior, the speaking probability dropping, or the mouth pausing; - how many passes the word has survived unchanged; - face visibility and EMG quality.

θ is a logistic regression trained on sessions 1 and 2. Every word on every pass is an example, labelled correct if it matches the true word at that position and never changed later.

A word locks when p_lock ≥ τ, where τ = clip(τ_0 − κ · predictability, τ_min, τ_max). Claude reports the predictability (0 to 1) with its prediction. τ_0 and κ are chosen on held-out data so that at most 2% of locked words are wrong, with the shortest delay. Claude can never lock a word alone: its feature is capped, and a hard rule requires camera–EMG agreement above a floor.

At the end of a sentence, the final pass rescores with the camera's attention beam (Auto-AVSR, beam 5), look-alike checks, the verifier and a Claude repair. A word still below τ then makes the screen show "Please repeat", with the features logged.

7. Evaluation

Benchmark	Data	Metric
Open vocabulary	Berkeley test set, 4 slots	WER, CER (greedy and beam + LM) vs `G4`
Homophene judge, 10 and 50 candidates	Berkeley test set	Top-1: can EMG pick the true sentence among lip look-alikes?

Benchmark	Data	Metric
Fast-speech judge	Berkeley test set compressed 1.25× and 1.5×	Top-1
Austin, session 3	100 held-out sentences, demo conditions	WER for camera alone, camera + EMG, camera + EMG + Claude; lock delay; wrong-lock rate; "Please repeat" rate
Speed	RTX 5060, on battery	Per-pass time for SVR-1, camera and decoder

Reference points: Gaddy and Klein report 76.8% of words wrong for their best 4 sites against 68.0% for all 8, on their audio-output metric. Our Gaddy-architecture retrain on 6 sites reached 33.2% WER (beam + LM) and 84.0% homophene-10. The 4-site numbers come from the `G4` and `SVR1_4ch` runs.

8. Changes from the current code

File	Change
`svr/preprocess.py`, `svr/data.py`	Replace the zero-phase notch + 2 Hz high-pass with `rig/filters.py` run in order; re-cache Berkeley to a new cache folder; add the scale statistics
`svr/svr1.py`	Add the speaking head (`Linear(512, 1)`, output `speak_logit`); keep `n_fixed = 0` (the fixed-sentence head is unused)
`svr/train_svr1.py`	Default `--keep-channels 0,1,2,5`; add the speaking loss, forced alignments every 10 epochs, and partial-sentence training; nothing closed-mouth remains
`svr/finetune.py`	Rig columns → slots `[2, 1, 5, 0]`; `rig/filters.py` and the scale factors instead of the old filter; drop the fixed-sentence head, "none" and closed-lips transfer; camera N-best negatives from recorded video; read the recorder's file format
`svr/recordings.py`	Load the recorder's 4-channel EMG and per-sample timestamps (format set with the recorder)
`svr/live.py` (new)	The live wrapper: ring buffer, causal filter state, scaling, re-read every 100 ms, outputs as in section 3
`fusion/decoder.py`, `fusion/lock.py` (new)	The joint decoder, gate and lock model of section 6
`bench/`	Run the benchmarks at 4 slots with the new preprocessing
`cluster/svr1_sweep.sh`	Main run `SVR1_4ch`, baseline `G4`; the old 6-site runs are retired
`tests/smoke_svr1_units.py`	4 slots, the speaking head, partial-sentence cuts
`figures/make_architecture.py`	The figure on this page